



BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

A PROJECT REPORT

ON

Live Mandi Price Advisor: A Real-Time Data-Driven Forecasting and Sell/Hold Advisory System for Indian Farmers

Submitted by

Tanmoy Maiti

Arnab Das

Raja Kumar

Aditya Bharati

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

**Computer Science & Engineering (Artificial Intelligence &
Machine Learning)**



BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

[Computer Science & Engineering (Artificial Intelligence & Machine Learning)]

BONAFIDE CERTIFICATE

Certified that this project report “**Live Mandi Price Advisor: A Real-Time Data-Driven Forecasting and Sell/Hold Advisory System for Indian Farmers**” is the bonafide work of “**Tanmoy Maiti, Arnab Das, Raja Kumar, Aditya Bharati**” who carried out the project work under my supervision.

SIGNATURE

Dr. Biswarup Mukherjee
HEAD OF THE DEPARTMENT
Professor & Head of the Department
Computer Science & Engineering (Artificial Intelligence & Machine Learning)
398, Ramkrishnapur Road, Barasat,
North 24 Parganas, Kolkata - 700 125

SIGNATURE

Dr. Biswarup Mukherjee
SUPERVISOR
Professor & Head of the Department
Computer Science & Engineering (Artificial Intelligence & Machine Learning)
398, Ramkrishnapur Road, Barasat,
North 24 Parganas, Kolkata - 700 125

Note: The Head of the Department and Project Supervisor are the same individual for this project. Both signature blocks are retained per the required certificate format.

Abstract

Small and marginal farmers in India routinely sell their produce without reliable knowledge of prevailing prices across nearby markets (mandis), leading to distress selling and avoidable income loss. This project, *Live Mandi Price Advisor: A Real-Time Data-Driven Forecasting and Sell/Hold Advisory System for Indian Farmers*, proposes a system that combines historical and live commodity price data from the Government of India's Agmarknet platform with time-series forecasting (Facebook Prophet) to generate short-term price forecasts and an actionable Sell/Hold recommendation for individual (commodity, mandi) pairs. The system is exposed through a two-page web application – an overview dashboard and an interactive, state-wise prediction interface – allowing farmers to compare prices across nearby markets before deciding when and where to sell. This report presents the problem motivation, a review of existing approaches, and the proposed system architecture, data pipeline, and machine learning design as developed to date.

Keywords: Agricultural price forecasting, Agmarknet, Prophet time-series model, distress selling, farmer decision support, mandi price comparison.

Contents

Bonafide Certificate	i
Abstract	ii
List of Tables	v
List of Figures	vi
List of Symbols, Abbreviations and Nomenclature	vii
1 Introduction	1
1.1 General	1
1.2 Problem Statement	1
1.3 Objectives	1
1.4 Scope of the Project	2
1.5 Organization of the Report	2
2 Literature Review	3
2.1 General	3
2.2 Commonly Proposed Approaches (and their limitations)	3
2.3 Data Sources for Indian Agricultural Prices	3
2.4 Forecasting Methodology	4
2.5 Positioning of the Present Work	4
3 Theory, Methodology, Materials & Methods	5
3.1 Proposed System Architecture	5
3.2 Data Pipeline	5
3.2.1 Historical Backfill	5
3.2.2 State and Crop Selection	6
3.2.3 Live Ingestion	6
3.2.4 Data Cleaning Considerations	6
3.3 Machine Learning Design	7
3.3.1 Task Formulation	7
3.3.2 Model Choice	7
3.3.3 State-Level Aggregation (not a separate model)	7
3.4 Technology Stack	7
3.5 Feasibility Discussion	8
3.5.1 Data Feasibility	8
3.5.2 Technical Feasibility	8
3.5.3 Timeline Feasibility	8

3.5.4	Ethical and Regulatory Considerations	8
4	Results, Analysis & Discussions	10
4.1	Status as of Review 1	10
4.2	Database Schema Implementation	10
4.3	Historical Backfill Results	10
4.4	Engineering Issue Encountered and Resolved	11
4.5	Discussion	11
5	Conclusion, Future Scope, Limitations	12
5.1	Conclusion (Current Stage)	12
5.2	Work Plan and Timeline	12
5.3	Current Limitations	13
5.4	Future Scope	13
A	Appendices	14
	Appendices	14
	Bibliography	15

List of Tables

3.1	Finalized state and top-6-crop selection, derived from reporting-consistency ranking on the 2024+2025 backfill data.	6
3.2	Finalized technology stack.	8
4.1	Historical backfill pipeline execution results.	10
4.2	Lookup table sizes after backfill.	11
5.1	Project phase-wise work plan and status as of Review 1.	12

List of Figures

3.1	Proposed system architecture: data sources, storage, model training/registry, API, and frontend.	5
-----	--	---

List of Symbols, Abbreviations and Nomenclature

Abbreviation	Full Form
ML	Machine Learning
API	Application Programming Interface
FAQ	Fair Average Quality (Agmarknet grading term)
MAE	Mean Absolute Error
CI	Continuous Integration
REST	Representational State Transfer
UI	User Interface
UX	User Experience
CRUD	Create, Read, Update, Delete

1.

Introduction

1.1 General

Agricultural markets (*mandis*) in India are numerous, fragmented, and geographically dispersed. A single commodity can trade at meaningfully different prices at mandis only a few hours apart on the same day, yet the individual farmer – especially small and marginal farmers who form the majority of India’s agricultural workforce – typically has no easy way to compare these prices before deciding where and when to sell. This information asymmetry, combined with limited on-farm storage capacity, frequently forces farmers into *distress selling*: selling immediately after harvest, at the point of lowest leverage, often to intermediaries who pay below the prevailing market rate.

This project, the **Live Mandi Price Advisor: A Real-Time Data-Driven Forecasting and Sell/Hold Advisory System for Indian Farmers**, addresses this gap directly. It combines publicly available government price data (Agmarknet, under the Ministry of Agriculture & Farmers Welfare, Government of India) with a time-series forecasting pipeline and a farmer-facing web application to give users a live, comparative, and forward-looking view of mandi prices for the crops and markets relevant to them.

1.2 Problem Statement

Given the absence of an accessible, forecast-aware, mandi-comparison tool for Indian farmers, this project aims to answer the following question: *Can publicly available agricultural market data be used to build a system that (a) forecasts short-term commodity prices at the level of individual mandis, and (b) presents this information, along with an actionable Sell/Hold recommendation, through an interface usable by a non-technical farmer?*

1.3 Objectives

The specific objectives of this project are:

1. To identify and consolidate reliable historical and live sources of Indian agricultural commodity price data (Agmarknet live API, and the Kaggle-compiled historical Agmarknet price dataset for backfill).
2. To design a data pipeline that ingests live daily price data on an ongoing basis while maintaining sufficient historical depth for time-series modelling.
3. To build a short-term price forecasting model at the granularity of individual (commodity, mandi) pairs, rather than a blurred state- or commodity-level

average, so that local price variation – the core value proposition of the system – is preserved.

4. To translate model output into a simple, actionable Sell/Hold signal that a farmer can interpret without statistical background.
5. To design and implement a two-page web application: a general dashboard (state-wise price overview) and an interactive prediction page allowing state- and crop-level drill-down to individual mandi comparisons.
6. To evaluate the forecasting approach against a naive baseline and document the comparison for academic assessment.

1.4 Scope of the Project

The system’s scope is finalized to three Indian states – Tamil Nadu, Uttar Pradesh, and Maharashtra – and the top six most-consistently-reported commodities per state (18 commodity-state pairs total), selected using actual historical reporting-frequency data from the 2024–2025 backfill rather than assumption, across roughly 15–20 mandis per state (Table 3.1, Chapter 3). This scope is intentionally bounded for a single academic-year capstone project; the architecture is designed to extend to additional states/commodities without structural change (see Chapter 3).

1.5 Organization of the Report

Chapter 2 reviews existing approaches to agricultural price prediction and related student/academic projects, and identifies the gap this project addresses. Chapter 3 presents the proposed system architecture, data pipeline, machine learning design, and technology stack, along with a feasibility discussion. Chapter 4 will present implementation results and evaluation (populated in subsequent review periods as development progresses). Chapter 5 presents the current work plan, timeline, and identifies the project’s present limitations and future scope.

2.

Literature Review

2.1 General

This chapter surveys existing work and commonly proposed student projects in agricultural machine learning, identifies which problem framings are already saturated, and positions the present project relative to that landscape.

2.2 Commonly Proposed Approaches (and their limitations)

A survey of recent student-project repositories, dataset platforms, and government-hackathon problem statements shows that a small number of problem framings recur very frequently in the "agricultural ML" space: crop yield prediction and crop-disease image classification (typically using the PlantVillage or similar Kaggle datasets), and generic price-prediction notebooks that report an accuracy or error metric without any accompanying usable interface. These approaches are well-documented and technically approachable, but by the same token are heavily repeated – reviewers and evaluators encounter near-identical submissions regularly, which reduces their value as a demonstration of original problem-solving.

Two structural weaknesses recur across this literature:

- **Granularity mismatch.** Many price/yield-prediction projects operate at a state or district level, which can mask meaningful local variation – the same crop can differ significantly in price between two mandis in the same state on the same day. A model trained at too coarse a granularity cannot support a genuinely local, actionable recommendation.
- **Missing "last mile".** Most academic implementations stop at a trained model and a reported accuracy figure, without an interface that would let a non-technical end user (a farmer, in this domain) actually act on the model's output.

2.3 Data Sources for Indian Agricultural Prices

The Government of India's **Agmarknet** portal (under data.gov.in) publishes daily wholesale/modal commodity prices from regulated mandis across the country, and is the primary live data source used by government and academic work in this space alike. For historical depth, this project uses the Kaggle-hosted "**Daily Market Prices of Commodity India**" dataset, compiled by the same maintainer as the live API dataset from the same underlying official Agmarknet records. Using one consistent data lineage across historical backfill and live collection avoids

the schema and coverage inconsistencies that can arise from stitching together independently-compiled sources.

2.4 Forecasting Methodology

Time-series forecasting of agricultural commodity prices has been approached in the literature using classical statistical methods (ARIMA and its variants), and more recently using additive decomposition models such as **Facebook Prophet**, which explicitly models trend and seasonal components and is well-suited to the kind of small, per-market datasets this project works with, without requiring extensive hyperparameter tuning. This project adopts Prophet as its primary forecasting model, benchmarked against a naive/moving-average baseline (see Chapter 3).

2.5 Positioning of the Present Work

Relative to the reviewed literature, this project is distinguished by: (a) forecasting at the individual (commodity, mandi) level rather than a coarser aggregate, preserving the local price variation that is the actual decision-relevant signal for a farmer; (b) combining a live, continuously updating data pipeline with a one-time historical backfill, rather than a single static training run; and (c) delivering the model’s output through a complete, usable web interface rather than stopping at a notebook-level result.

3.

Theory, Methodology, Materials & Methods

3.1 Proposed System Architecture

The system is organized into five stages: historical data backfill, live daily ingestion, model training/retraining, a backend API layer, and a two-page frontend application. Figure 3.1 illustrates the overall data and control flow.

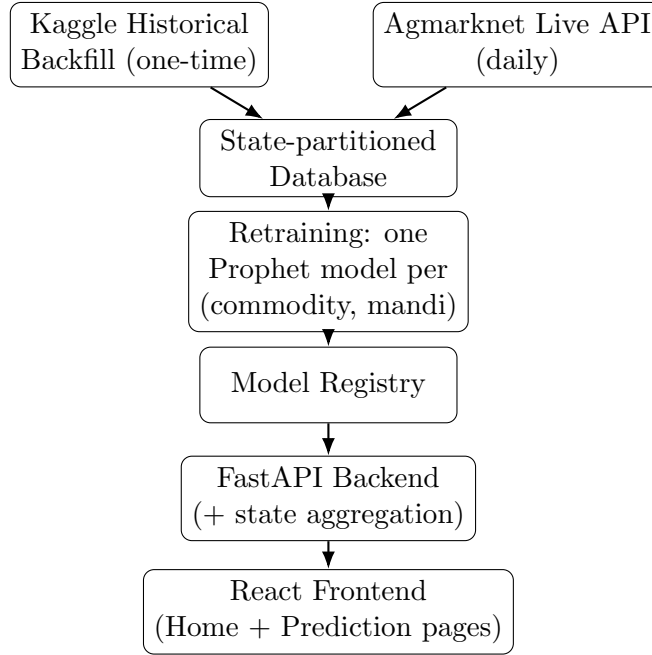


Figure 3.1: Proposed system architecture: data sources, storage, model training/registry, API, and frontend.

An important design decision, discussed further in Section 3.4, is that **storage and the user interface are organized by state**, while the **forecasting model itself remains at (commodity, mandi) granularity** – state-level views are computed as an aggregation over underlying mandi-level forecasts rather than being separately modelled.

3.2 Data Pipeline

3.2.1 Historical Backfill

Prophet requires several months of history to reliably detect weekly and yearly seasonality; this cannot be obtained from live daily collection alone within a single academic year. The system therefore performs a one-time historical backfill from the Kaggle "Daily Market Prices of Commodity India (2001–2026)" dataset – specifically its 2024 and 2025 slices – before daily live collection begins. This

dataset is compiled by the same original maintainer as the live Agmarknet API dataset used for daily ingestion, giving one consistent data lineage across historical and live collection.

3.2.2 State and Crop Selection

The three states and six crops per state (Table 3.1) were finalized by ranking commodities within the 2024+2025 backfill data on reporting consistency (days/markets reported), not raw row count alone, so that a commodity with sporadic gaps is not favoured over one with steady daily coverage.

State	Top 6 crops (ranked)
Tamil Nadu	Coconut; Bhindi (Ladies Finger); Green Chilli; Bottle gourd; Snakeguard; Onion (edged out Banana-Green by a narrow margin, ranked on combined 2024+2025 data)
Uttar Pradesh	Potato; Onion; Tomato; Wheat; Brinjal; Green Chilli (stable ranking independently in both 2024 and 2025)
Maharashtra	Wheat; Bengal Gram; Soyabean; Arhar (Tur/Red Gram); Onion; Jowar (Sorghum) (stable ranking independently in both 2024 and 2025)

Table 3.1: Finalized state and top-6-crop selection, derived from reporting-consistency ranking on the 2024+2025 backfill data.

3.2.3 Live Ingestion

A scheduled job (GitHub Actions cron) fetches the previous day’s records from the Agmarknet API (resource ID 9ef84268-d588-465a-a308-a864a43d0070) and appends them to the same state-partitioned tables used by the historical backfill, so that the model’s training set grows continuously.

3.2.4 Data Cleaning Considerations

Preliminary analysis of a sample Agmarknet export surfaced two practical issues that the ingestion pipeline must handle:

- **Field-name encoding.** Exported column names contain an XML-encoding artefact (e.g. `Min_x0020_Price`) rather than the clean `min_price` field name returned by the JSON API; the pipeline must normalize this during ingestion.
- **Variety/grade duplication.** A single (market, commodity, date) combination can have multiple rows if more than one variety/grade of that commodity was traded that day (confirmed empirically, e.g. multiple *Areca nut* rows at the same market on the same day with different modal prices). The pipeline filters to a consistent variety/grade per (commodity, mandi) series before constructing the Prophet training set.

3.3 Machine Learning Design

3.3.1 Task Formulation

The core forecasting task is **time-series regression**, not classification: the model predicts a continuous value (the next-period modal price), using `modal_price` as the target variable (the price at which the largest transacted quantity occurred that day, per Agmarknet convention) rather than the min/max range. The farmer-facing Sell/Hold/Stable label is a rule-based decision layer applied on top of the continuous forecast, not a directly trained classifier – this preserves the underlying magnitude/confidence information that a bare label would discard, and mirrors how real trading/advisory systems are typically structured.

3.3.2 Model Choice

Facebook Prophet (an additive decomposition model: trend + weekly seasonality + yearly seasonality) is used as the primary forecasting model, trained independently per (commodity, mandi) pair. A naive/moving-average baseline is trained alongside it for comparison, evaluated using Mean Absolute Error (MAE), to provide an evidence-based justification for the modelling choice rather than an assumed one.

3.3.3 State-Level Aggregation (not a separate model)

The Prediction page’s state-level summary view (“select a state, see its top 6 crops with percentage change”) is computed by aggregating the outputs of the existing per-mandi models – e.g. averaging the forecast percentage change across all mandis reporting a given commodity within the selected state – rather than training a new, coarser model. This preserves per-mandi precision for the underlying comparison feature while still providing a fast, simple state-level overview.

3.4 Technology Stack

Table 3.2 summarizes the finalized technology choices and their justification.

Layer	Technology	Justification
Historical back-fill	Kaggle “Daily Market Prices of Commodity India” (2024+2025 parquet)	Same lineage/maintainer as live API dataset; bulk-filterable across states/crops
Live data source	data.gov.in Agmarknet API	Free, structured, government-maintained
Forecasting model	Facebook Prophet	Handles small per-mandi series well; minimal tuning
Baseline model	Naive / moving average (scikit-learn)	Enables an evidence-based accuracy comparison

Layer	Technology	Justification
Backend	FastAPI (Python)	Lightweight, async, auto-documented REST API
Database	SQLite (dev) / Supabase Postgres (prod)	Free hosting; state-partitioned schema
Frontend	React (Vite)	Required for interactive, clickable India map
Mapping	react-simple-maps + TopoJSON	Standard approach for clickable choropleth/state maps
Charting	recharts	React-native price/forecast visualization
Scheduling	GitHub Actions	Free CI/CD-based cron scheduling
Deployment	Vercel/Netlify (frontend), Render (backend), Supabase (DB)	All free-tier, standard for student deployment

Table 3.2: Finalized technology stack.

3.5 Feasibility Discussion

3.5.1 Data Feasibility

Both the live (Agmarknet API) and historical (Kaggle-compiled Agmarknet) data sources have been confirmed to be freely accessible without institutional gate-keeping, unlike domains such as healthcare (patient-data access restrictions) or finance (proprietary transaction data), which was a key factor in selecting this domain over alternatives originally considered.

3.5.2 Technical Feasibility

Each component of the proposed stack (Prophet, FastAPI, React) is well-documented and widely used in comparable applications, keeping the implementation risk manageable for a team building most of this tooling for the first time in a practical setting.

3.5.3 Timeline Feasibility

The project scope (2–3 states, six commodities per state, roughly 15–20 mandis per state) was deliberately bounded to fit a single academic-year capstone timeline, with a clear path to widening coverage later without architectural change (Section 3.1).

3.5.4 Ethical and Regulatory Considerations

As the system uses only publicly published government price data and does not process any personal, medical, or financial data, it does not raise the consent, pri-

vacy, or regulatory concerns that a healthcare- or finance-domain project would.

4.

Results, Analysis & Discussions

4.1 Status as of Review 1

The historical data foundation for the project has been implemented and validated end-to-end: a normalized PostgreSQL (Supabase) schema is live, and the full 2024–2025 Kaggle historical backfill has been loaded, cleaned, and verified against it. Live data ingestion, forecasting model training, and the frontend application remain in progress and had not yet produced evaluable output at the time of this submission.

4.2 Database Schema Implementation

The schema described in Chapter 3 was implemented using SQLAlchemy Core against Supabase PostgreSQL: five normalized lookup tables (**states**, **districts**, **markets**, **commodities**, **varieties**) respecting a strict hierarchy (district scoped by state, market scoped by district, variety scoped by commodity – a market or variety name is not assumed unique on its own), plus **price_records** (the single fact table, not partitioned by state), **forecasts**, and **model_registry**. The connection and schema creation were verified independently before any data loading was attempted.

4.3 Historical Backfill Results

Table 4.1 summarizes the actual execution of `backfill_kaggle.py` against the real 2024.parquet and 2025.parquet files, run on 11 September 2026.

Stage	Result
Raw combined rows (2024 + 2025)	11,363,982
Filtered to the 18 finalized state-crop combinations	1,208,648
Invalid price cells (≤ 0 , converted to missing)	629
Structurally invalid rows (Min > Max, or Modal > Max)	13
Extreme anomaly rows removed	27
Duplicate natural-key groups collapsed	22,950 (22,951 rows)
Final rows upserted into price_records	1,185,657
Date range in database	2024-01-01 to 2025-12-29

Table 4.1: Historical backfill pipeline execution results.

Post-load validation confirmed: zero rows with invalid or missing `modal_price`,

and zero duplicate natural-key groups in the database – i.e. the uniqueness constraint on (market, commodity, variety, arrival date) was never violated across the full load. Table 4.2 shows the resulting lookup table sizes.

Lookup table	Row count
states	3
districts	143
markets	1,372
commodities	14
varieties	75

Table 4.2: Lookup table sizes after backfill.

Notably, the 18 finalized state-crop combinations resolve to only 14 distinct `commodities` rows, not 18 – because Onion (reported in all three states), Wheat (Uttar Pradesh and Maharashtra), and Green Chilli (Tamil Nadu and Uttar Pradesh) are the same commodity row shared across states, correctly normalized rather than duplicated. Supabase storage usage after the full load was approximately 0.11 GB of the 2 GB free-tier provision, confirming no near-term storage concern for the current project scope.

4.4 Engineering Issue Encountered and Resolved

The first full backfill run used a 5,000-row batch size for the PostgreSQL upsert and failed partway through, most likely due to a statement-size or connection-duration constraint on Supabase’s free tier (the exact server-side cause was not confirmed from the client traceback). Because the entire backfill runs inside a single database transaction, the partial run was safely rolled back rather than leaving inconsistent data. Reducing the batch size to 1,000 rows resolved the issue without any other pipeline changes, and the corrected run completed successfully with the results above.

4.5 Discussion

This stage validated several Chapter 3 design decisions directly against real data rather than assumption: the hierarchical (not flat) lookup-ID mapping was necessary in practice (market and variety names do recur across different parent districts/commodities in the raw data), and the originally-scoped cleaning rules (invalid prices, natural-key deduplication) were insufficient on their own – structurally invalid rows and extreme statistical outliers (e.g. a single recorded modal price of Rs. 917,588,483 for onion) required additional, explicitly-documented cleaning rules not anticipated in the original design. This is treated as expected refinement through implementation, consistent with this report’s approach throughout: design decisions are validated or revised against real data as it becomes available, rather than assumed correct from documentation alone.

5.

Conclusion, Future Scope, Limitations

5.1 Conclusion (Current Stage)

This report has presented the motivation, problem statement, literature positioning, and complete proposed architecture for the Live Mandi Price Advisor: A Real-Time Data-Driven Forecasting and Sell/Hold Advisory System for Indian Farmers. The system’s design – a state-organized but mandi-granular forecasting approach, combining a one-time historical backfill with a continuously updating live pipeline – was refined through preliminary analysis of real Agmarknet data rather than fixed in advance from assumption alone. The project’s codebase has been scaffolded in line with this design and is under active development.

5.2 Work Plan and Timeline

Table 5.1 presents the project’s phase-wise plan and current status as of Review 1. Status will be updated at each subsequent review period.

Phase	Deliverable	Status
1	Supabase connection verified, normalized DB schema (single <code>price_records</code> fact table, not state-partitioned), Kaggle parquet historical backfill loaded and validated (1,185,657 records, 3 states, 2024-01-01 to 2025-12-29 – Section 3.2.2)	Done
2	Data-driven top-6-crops-per-state selection (Table 3.1, Chapter 3)	Done
3	Prophet model + naive baseline per (commodity, mandi), MAE comparison	Not Started
4	Sell/Hold decision logic, validated against historical cases	Not Started
5	Retraining pipeline, model registry, state-aggregation logic	Not Started
6	FastAPI backend: per-mandi and state-summary endpoints	Not Started
7	React frontend: Home page	In Progress
8	React frontend: Prediction page (map + drill-down)	Not Started
9	Automated scheduling and deployment	Not Started
10	Polish, documentation, final report	Not Started

Table 5.1: Project phase-wise work plan and status as of Review 1.

5.3 Current Limitations

As the project is at an early implementation stage, the following limitations apply as of this review and are expected to be resolved in subsequent phases:

- No trained forecasting model exists yet; forecasting accuracy cannot be evaluated at this stage (planned for Phase 3).
- The live data pipeline and scheduled retraining job are currently stub implementations (intentionally not yet scheduled to run automatically, to avoid failures on incomplete code).
- The frontend currently covers only the Home page structure; the interactive India-map prediction page is planned for Phase 8.
- Whether multiple reported varieties of the same commodity (e.g. Onion, Green Chilli) should be collapsed into a single series per (commodity, mandi) or modelled separately per variety is not yet decided; this affects total model count and will be resolved before Phase 3 training begins.

5.4 Future Scope

Beyond the current planned scope, the architecture supports several natural extensions without structural redesign: additional states and commodities (Section 3.1), explainability layers (e.g. SHAP/LIME) for the forecasting model to support academic evaluation and farmer trust, SMS/regional-language delivery of Sell/Hold alerts for farmers without reliable smartphone/internet access, and integration of weather data as an additional forecasting feature.

A.

Appendices

A.1 Repository

The project's source code and this report's LaTeX source are maintained at:
<https://github.com/live-mandi-advisor/live-mandi-price-advisor>

Bibliography

- [1] Directorate of Marketing and Inspection, Ministry of Agriculture and Farmers Welfare, Government of India. Agmarknet: Agricultural marketing information network. <https://agmarknet.gov.in>, 2026.
- [2] Kaggle. Daily market prices of commodity india (2001–2026). <https://www.kaggle.com>, 2026.
- [3] Sean J. Taylor and Benjamin Letham. Forecasting at scale. *The American Statistician*, 72(1):37–45, 2018.